

Terraform_2 Task

Why Terraform is important

1. **Infrastructure as Code (IaC)**
Write infrastructure in .tf files → version-controlled, repeatable, and auditable.
2. **Automation & Speed**
Create servers, networks, databases, and services in minutes with init → plan → apply.
3. **Consistency & No Human Error**
Same code = same infrastructure across dev, test, and prod.
4. **Multi-Cloud Support**
Works with AWS, Azure, GCP, and on-prem from a **single tool**.
5. **State Management**
Tracks real infrastructure using a **state file**, enabling safe updates and drift detection.
6. **Idempotency**
Running Terraform multiple times won't break infra — it only changes what's needed.
7. **Easy CI/CD Integration**
Integrates smoothly with Jenkins, GitHub Actions, GitLab CI, etc.
8. **Scalable & Modular**
Reuse infrastructure using **modules** as your environment grows.
9. **Cost & Resource Control**
Plan before apply → know exactly what will be created or destroyed.
10. **Industry Standard**
Widely used in DevOps and cloud roles; strong interview and production relevance.

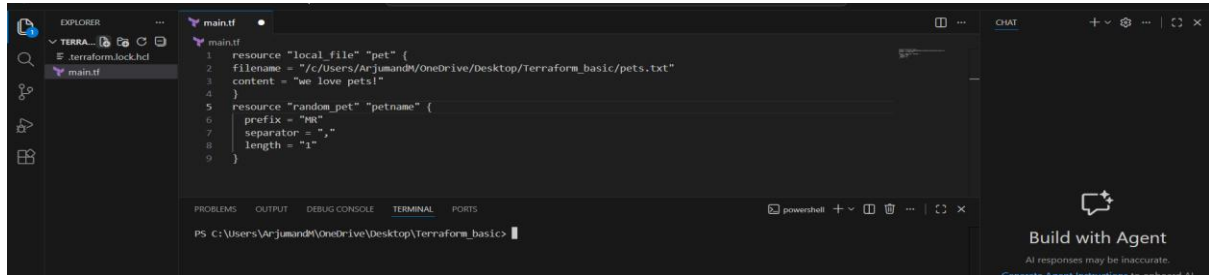
One-line interview answer

Terraform is important because it automates infrastructure provisioning using code, ensuring consistency, scalability, and multi-cloud support with safe, repeatable deployments.

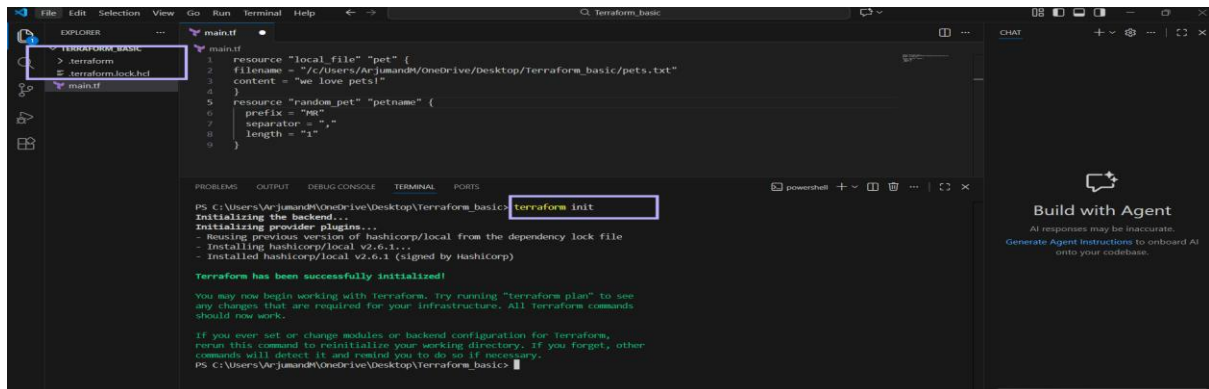
Execute all the templates shown in video.

1. Note down below points, Terraform Init Terraform Plan Terraform Apply Terraform Provider.

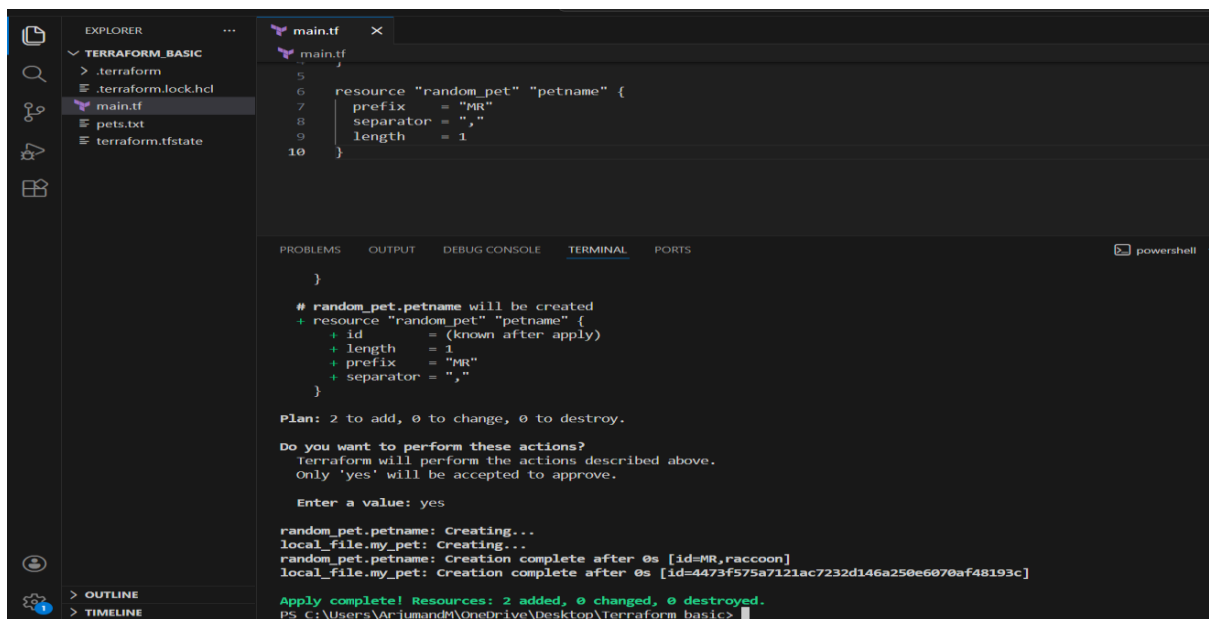
- Open visual studio and open main.tf file which we created in last task
- As we see we are in main.tf file and destroyed previous file



- Type terraform init it will initiate and create a backup to .terraform file as we can see



- After passing terraform apply command it will create terraform.tfstate file, it is blue print, it contain the the metadata.



- When we make changes hen instead of cat, it will show the changes because all the records will be stored in terraform.tfstate file, from that file terraform will know the changes

```

1 resource "local_file" "my_pet" {
2   filename = "pets.txt"
3   content  = "my cat name is hen"
4 }
5
6 resource "random_pet" "petname" {
7   prefix      = "MR"
8   separator   = "-"
9   length      = 1
10 }

```

```

Terraform will perform the following actions:

# local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
  ~ content          = "my cat name is CAT" -> "my cat name is hen" # forces replacement
  ~ content_base64sha256 = "kjTj2FFJssQClvfV8pxh2tVUKTFX6SPcrGd9NMPcdY=" -> (known after apply)
  ~ content_base64sha512 = "X4SM1rKXsPnWcQ4wGX799VOFBPAEBgy9+nIGNZa5JWFLGet8vUUBoHSD2uIArgJKVqihw8XSac1+G6qLhkCw==" -> (known after apply)
  ~ content_md5         = "363c9420b9aa2e84ac432171058bf39" -> (known after apply)
  ~ content_sha1        = "4473f575a7121ac7232d146a250e6070af48193c" -> (known after apply)
  ~ content_sha256      = "9234e3d85149b2c40296f7d5f29c61dabbd4293157e923dcae00fd3663c171d6" -> (known after apply)
  ~ content_sha512      = "5f848cd6b28cb0c9d6710e305c65fbf7d54e1413c011b1b2f7e9c818d65ae495852c67adf2fb9405ba0749ddae200ae024abe1cc5269cd7e1baa8b8640b" -> (known after apply)
  ~ id                  = "4473f575a7121ac7232d146a250e6070af48193c" -> (known after apply)
  # (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

```

- If we go to terraform.tfstate file we can see the records along with changes

```

1 {
2   "resources": [
3     {
4       "schema_version": 0,
5       "attributes": {
6         "content": "my cat name is hen",
7         "content_base64": null,
8         "content_base64sha256": "8cFFyVnyuvtfwIN9xw+HdNv/kEGYiovgdt.SHAHfY5pw=",
9         "content_base64sha512": "NNkDKb0pndFtd3aj/wDZikuJ4QhndWwNRWn6hcssgsxjo4WzK/2qRBCr90ryyNPAe/DJvWE",
10        "content_md5": "c86b349e4c9c97af30ba1895e9bb9aa5",
11        "content_sha1": "2737d7fbd971d5885dc98b8eebb2d53f7666595f",
12        "content_sha256": "5c042717a9f9ae4b64eb918f",
13        "content_sha512": "5c042717a9f9ae4b64eb918f"
14      }
15    }
16  ]
17 }

```

- If we are making changes by default it will destroy and then create as we can see.

```

1 resource "local_file" "my_pet" {
2   filename = "pets.txt"
3   content  = "my cat name is dog"
4 }
5
6 resource "random_pet" "petname" {
7   prefix      = "MR"
8   separator   = "-"
9   length      = 1
10 }

```

```

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.my_pet: Destroying... [id=2737d7fbd971d5885dc98b8eebb2d53f7666595f]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=35830f5f9e0980d8e52857af699c90650e8751b]

```

- By using lifecycle rule we can change that default changes we can ask change the role and add rule as create first before destroy,
- Following line we can add in function, lifecycle { \create_before_destroy =true \ }
- After adding that line save the file and then pass command 'terraform apply'

```

main.tf
1 resource "local_file" "my_pet" {
2   filename = "pets.txt"
3   content  = "my cat name is Tiger"
4   lifecycle {
5     create_before_destroy = true
6   }
7 }
8
9 resource "random_pet" "petname" {
10  prefix = "MR"
11  separator = "-"
12  length = 4
13 }

r apply)
~ content_md5      = "03b91d56ce2a0ad7bb1434ea1ad60197" -> (known after apply)
~ content_sha1     = "35830f5f9e098dd0e52857af699c90650e87571b" -> (known after apply)
~ content_sha256   = "751c3d4b8a098418e73821b3d5aaabcbac433b462325447fc5f4fe300af7d73e" -> (known after apply)
~ content_sha512   = "099ed0276796895794aba774b48e5942438547afb31b58e2c0e539ab899480e5b416a1d5b8399be7dbef532965fbb38879d3ae
c12ccacbe68b1cf0e9ed6a5" -> (known after apply)
~ id              = "35830f5f9e098dd0e52857af699c90650e87571b" -> (known after apply)
# (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=ab565fe2abebf8be2b4cebf576274cf010721f5]
local_file.my_pet (deposed object 707792ce): Destroying... [id=35830f5f9e098dd0e52857af699c90650e87571b]
local_file.my_pet: Destruction complete after 0s

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

```

- We can also prevent from destroy as we can see in the below
- I have used following command 'lifecycle { \ prevent_destroy = true \ }'

```

main.tf
1 resource "local_file" "my_pet" {
2   filename = "pets.txt"
3   content  = "my cat name is rabbit"
4   lifecycle {
5     prevent_destroy = true
6   }
7 }
8
9 resource "random_pet" "petname" {
10  prefix = "MR"
11  separator = "-"
12  length = 4
13 }

r apply)
~ content          = "my cat name is Tiger" -> "my cat name is rabbit" # forces replacement
~ content_base64sha256 = "R1N6Jm8soTJUG+9RC00770aH4B5DWAfR4andRd4P28A-" -> (known after apply)
~ content_base64sha512 = "S8FK26XzrvorbfjwcmiQLx0pfowA3vKESp9a8bCg10wpvEH4P2Swrfu0b0vL0H4A/afiZjNSPh75K2mthg==" -> (known after apply)
~ content_md5       = "8487403389a52b9202c47ae69409a679" -> (known after apply)
~ content_sha1      = "ab565fe2abebf8be2b4cebf576274cf010721f5" -> (known after apply)
~ content_sha256    = "47537a270af4b284c953afbd44e3bbe8687b81e48580151e1a9dd45df8f67c8" -> (known after apply)
~ content_sha512    = "4bc14acfa5f3aefa1badf8d6f9cbe7890325c7da5fa30037be4112a7d6bc6f0aa0d74c29f9e1cfd25af7d475b874fb3a1e0efd
a7fe6635923e1ef92b6c2686" -> (known after apply)
~ id               = "ab565fe2abebf8be2b4cebf576274cf010721f5" -> (known after apply)
# (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Error: Instance cannot be destroyed
on main.tf line 1:
1: resource "local_file" "my_pet" {

```

- We can also add ignore_changes as we can see in below image

```

1 resource "local_file" "my_pet" {
2   filename = "pets.txt"
3   content  = "my pet name is hen"
4   lifecycle {
5     ignore_changes = [
6       content
7     ]
8   }
9 }
10
11 resource "random_pet" "petname" {
12   prefix = "MR"
13 }

```

```

PS C:\Users\Varjun\OneDrive\Desktop\Terraform_basics> terraform apply
local_file.my_pet: Refreshing state... [id=ab565fe2abebf8be2b4cebf576274cf010721f5]
random_pet.petname: Refreshing state... [id=MR,raccoon]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
PS C:\Users\Varjun\OneDrive\Desktop\Terraform_basics> terraform apply
local_file.my_pet: Refreshing state... [id=ab565fe2abebf8be2b4cebf576274cf010721f5]
random_pet.petname: Refreshing state... [id=MR,raccoon]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
PS C:\Users\Varjun\OneDrive\Desktop\Terraform_basics>

```

- we can variable to our file to avoid hardcoding
- Go to top and create a new file and then add variable file with .tf extension

```

1 variable "filename" {
2   default = "pets.txt"
3   type    = string
4 }

```

```

PS C:\Users\Varjun\OneDrive\Desktop\Terraform_basics>

```

- As we can see we have added variable.tf in that we have provided variable

```

1 resource "local_file" "my_pet" {
2   filename = var.filename
3   content  = var.content
4 }
5
6 resource "random_pet" "petname" {
7   prefix = "MR"
8   separator = "-"
9   length = 1
10 }

```

```

r apply)
~ content_md5      = "8487403389a52b9202c47ae69409a679" -> (known after apply)
~ content_sha1     = "ab565fe2abebf8be2b4cebf576274cf010721f5" -> (known after apply)
~ content_sha256   = "47537a2708af2ab284c053afbd44e3bbe0868781e48580151e1a9dd45df8f67ce" -> (known after apply)
~ content_sha512   = "4bc14acfa5f3aef1badf9cbe7890325c7da5fa30037be4112a7d6bc6f00a0d74c29f9e1cfd25af7d475b874f8b3a1e00fd
a7fe635923e1ef92b6c2686" -> (known after apply)
~ id               = "ab565fe2abebf8be2b4cebf576274cf010721f5" -> (known after apply)
# (3 unchanged attributes hidden)

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.my_pet: Destroying... [id=ab565fe2abebf8be2b4cebf576274cf010721f5]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=29f716260a7733eb0ecd1083b6e0d754c8de1ffc]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

```

- We have added variables for all 4 resources filename, content, prefix, length

The screenshot shows the VS Code interface with the following files open:

- main.tf**:


```

5
6 resource "random_pet" "petname" {
7   prefix = var.prefix
8   separator = ","
9   length = var.length
10 }
      
```
- variables.tf**:


```

1 variable "filename" {
2   default = "pets.txt"
3   type = string
4 }
5 variable "content" {
6   default = "Rabbit"
7   type = string
8 }
9 variable "prefix" {
10  default = "Arjumand"
11  type = string
12 }
13 variable "length" {
14  default = "25"
15  type = number
16 }
      
```
- pets.txt**: (Content not visible)

- We can also provide variable values in power cell

The screenshot shows the VS Code interface with the following files open:

- main.tf**:


```

1 resource "local_file" "my_pet" {
2   content = var.content
3 }
4
5
6 resource "random_pet" "petname" {
7   prefix = "MR"
8   separator = ","
9   length = "1"
10 }
      
```
- variables.tf**:


```

1 variable "filename" {
2   }
3 }
4 variable "content" {
5   }
6 }
      
```
- wild.txt**: (Content not visible)

- I have provided variable in the PowerShell

The screenshot shows a PowerShell terminal window with the following output:

```

PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform basic> terraform apply -var "filename=wild.txt" -var "content=i love rabbit"
local_file.my_pet: Refreshing state... [id=29f716260a7733eb00cd1083b6e0d754c8d01ffc]
random_pet.petname: Refreshing state... [id=MR,destined,cowbird]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
-/+ destroy and then create replacement
  
```

- Variable is added sucessfully

The screenshot shows a PowerShell terminal window with the following output:

```

}

Plan: 2 to add, 0 to change, 2 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

random_pet.petname: Destroying... [id=MR,destined,cowbird]
random_pet.petname: Destruction complete after 0s
local_file.my_pet: Destroying... [id=29f716260a7733eb00cd1083b6e0d754c8d01ffc]
local_file.my_pet: Destruction complete after 0s
random_pet.petname: Creating...
local_file.my_pet: Creating...
random_pet.petname: Creation complete after 0s [id=MR,hare]
local_file.my_pet: Creation complete after 0s [id=22136d30b33e1e57feebcc6cec21aef87f2ae2b4]

Apply complete! Resources: 2 added, 0 changed, 2 destroyed.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform basic>
  
```

```

PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> terraform apply
random_pet.petname: Refreshing state... [id=MR,raccoon]
local_file.my_pet: Refreshing state... [id=29f716260a7733eb0cd1083b6e0d754c8d01ffc]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # random_pet.petname must be replaced
-/+ resource "random_pet" "petname" {
  ~ id      = "MR,raccoon" -> (known after apply)
  ~ length  = 1 -> 2 # forces replacement
    # (2 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

  ~ id      = "MR,raccoon" -> (known after apply)
  ~ length  = 1 -> 2 # forces replacement
    # (2 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.

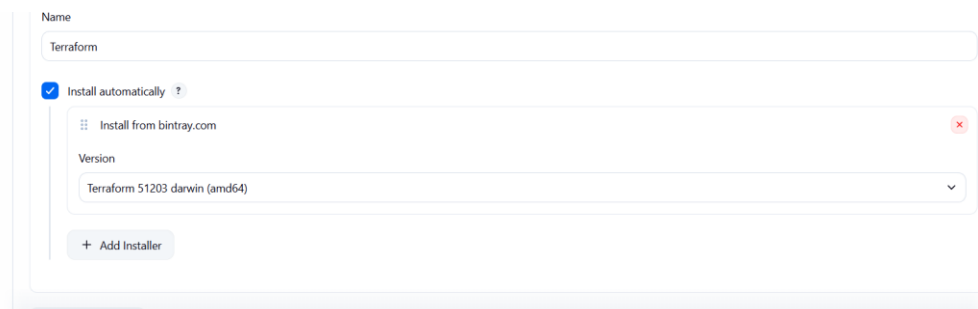
```

Ln 10, C

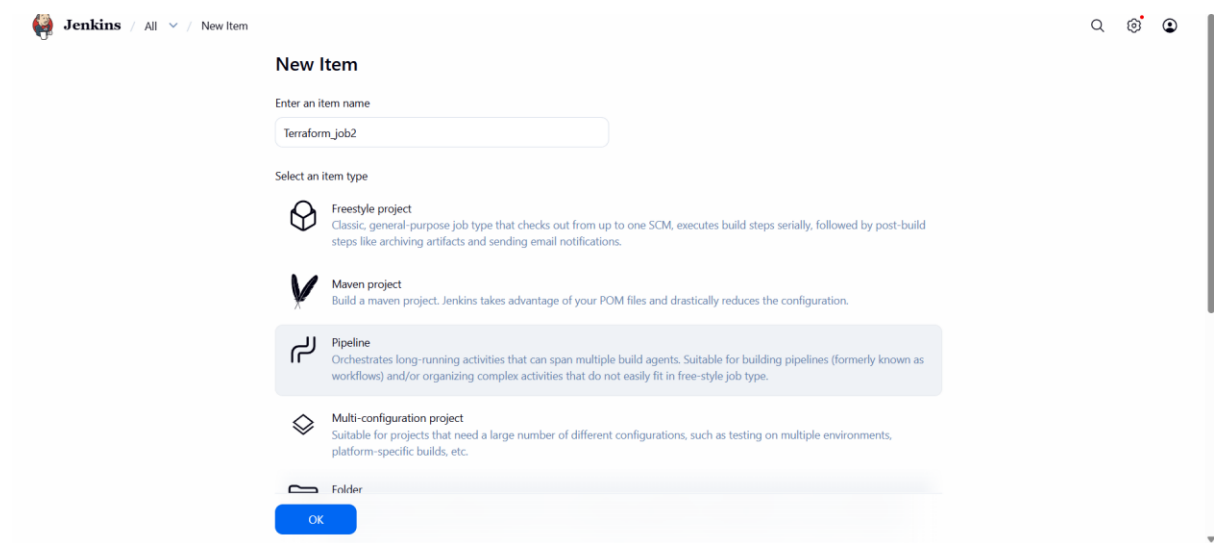
2. Integrate a sample Terraform template in Jenkins.

1. Install **Terraform Plugin** from *Manage Jenkins* → *Manage Plugins*.
2. Install **Terraform CLI** on the Jenkins server.
3. Verify Terraform using `terraform version`.
4. Go to *Manage Jenkins* → *Global Tool Configuration*.
5. Add **Terraform** tool and set Terraform binary path.
6. Create a **Pipeline job** in Jenkins.
7. Connect the job to your **Terraform Git repository**.
8. Add stages for `terraform init`, `validate`, `plan`, and `apply`.
9. Use `sh` (Linux) or `bat` (Windows) to run Terraform commands.
10. Save the job and **build the pipeline** to execute Terraform.

- After Installing the terraform plugins, go to tools and terraform



Create a new job and select pipeline and create OK



Jenkins / All / New Item

New Item

Enter an item name

Terraform_job2

Select an item type

-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
-  **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
-  **Folder**

OK

```
pipeline {
```

```
  agent any
```

```
  stages {
```

```
    stage('Checkout SCM') {
```

```
      steps {
```

```
        git branch: 'main',
```

```
        url: 'https://github.com/arjumand9794/terraform.git'
```

```
      }
```

```
    }
```

```
    stage('Terraform Init') {
```

```
      steps {
```

```
        sh 'terraform init'
```

```
      }
```

```
    }
```



```
stage('Terraform Validate') {  
    steps {  
        sh 'terraform validate'  
    }  
}
```

```
stage('Terraform Plan') {  
    steps {  
        sh 'terraform plan'  
    }  
}
```

```
stage('Terraform Apply') {  
    steps {  
        sh 'terraform apply -auto-approve'  
    }  
}  
}
```

Create a pipeline in the script

Configure

- General
- Triggers
- Pipeline
- Advanced

Pipeline script

Script ?

```
1 pipeline {  
2     agent any  
3  
4     stages {  
5         stage('Terraform Init') {  
6             steps {  
7                 sh 'terraform init'  
8             }  
9         }  
10  
11        stage('Terraform Plan') {  
12            steps {  
13                sh 'terraform plan'  
14            }  
15        }  
16    }  
17 }
```

☒ Use Groovy Sandbox ?

Pipeline was created successfully with my repo which has a file .tf file in it

